

Отчёт по лабораторной работе № 5

Название работы: Модель ветвления Git Flow

Студент: ДУРАК РАМАЗАН УГУР

Группа: AT-15

Дата выполнения: 16.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Инициализировать репозиторий, создать ветку develop от main	☒	<pre>\$ git init Initialized empty Git repository in C:/Users/ugur/lab5_gitflow/.git / \$ echo "# Git Flow" > README.md \$ git add . \$ git commit -m "Initial main" [main (root-commit)] Initial main 1 file changed, 1 insertion(+) create mode 100644 README.md \$ git checkout -b develop Switched to a new branch 'develop'</pre>
2	Создать feature-ветку login от develop, добавить auth.cfg, смержить с --no-ff	☒	<pre>\$ git checkout -b feature/login develop Switched to a new branch 'feature/login' \$ echo "login: admin" > auth.cfg \$ git add . \$ git commit -m "Add auth</pre>

Шаг	Описание	Вып.	Команды / вывод
			<pre>config" [feature/login] Add auth config 1 file changed, 1 insertion(+) create mode 100644 auth.cfg \$ git checkout develop Switched to branch 'develop' \$ git merge --no-ff feature/login -m "Merge login feature" Merge made by the 'ort' strategy. auth.cfg 1 + 1 file changed, 1 insertion(+) create mode 100644 auth.cfg</pre>
3	Создать feature-ветку db от develop, добавить db.cfg, сменить с --no-ff	✓	<pre>\$ git checkout -b feature/db develop Switched to a new branch 'feature/db' \$ echo "db_host=localhost" > db.cfg \$ git add . \$ git commit -m "Add DB config" [feature/db] Add DB config 1 file changed, 1 insertion(+) create mode 100644 db.cfg \$ git checkout develop Switched to branch 'develop' \$ git merge --no-ff feature/db - m "Merge DB feature" Merge made by the 'ort' strategy. db.cfg 1 + 1 file changed, 1 insertion(+) create mode 100644 db.cfg</pre>
4	Создать release-ветку v1.0 от develop, добавить version.txt, сделать коммит	✓	<pre>\$ git checkout -b release/v1.0 develop Switched to a new branch 'release/v1.0' \$ echo "version=1.0" > version.txt \$ git add . \$ git commit -m "Bump to 1.0"</pre>

Шаг	Описание	Вып.	Команды / вывод
			[release/v1.0] Bump to 1.0 1 file changed, 1 insertion(+) create mode 100644 version.txt
5	Слить release в main (с --no-ff), создать тег v1.0	✓	\$ git checkout main Switched to branch 'main' \$ git merge --no-ff release/v1.0 -m "Release v1.0" Merge made by the 'ort' strategy. version.txt 1 + 1 file changed, 1 insertion(+) create mode 100644 version.txt \$ git tag -a v1.0 -m "Version 1.0"
6	Слить release обратно в develop (back-merge) с --no-ff	✓	\$ git checkout develop Switched to branch 'develop' \$ git merge --no-ff release/v1.0 -m "Back-merge release" Merge made by the 'ort' strategy. version.txt 1 + 1 file changed, 1 insertion(+) create mode 100644 version.txt
7	Создать hotfix-ветку critical от main, добавить patch.log, сделать коммит	✓	\$ git checkout main Switched to branch 'main' \$ git checkout -b hotfix/critical main Switched to a new branch 'hotfix/critical' \$ echo "patch applied" > patch.log \$ git add . \$ git commit -m "Critical security fix" [hotfix/critical 52da0e6] Critical security fix 1 file changed, 1 insertion(+)

Шаг	Описание	Вып.	Команды / вывод
			create mode 100644 patch.log
8	Слить hotfix в main (с --no-ff), создать тег v1.0.1	✓	\$ git checkout main Switched to branch 'main' \$ git merge --no-ff hotfix/critical -m "Hotfix critical" Merge made by the 'ort' strategy. patch.log 1 + 1 file changed, 1 insertion(+) create mode 100644 patch.log \$ git tag v1.0.1
9	Слить hotfix в develop (с --no-ff)	✓	\$ git checkout develop Switched to branch 'develop' \$ git merge --no-ff hotfix/critical -m "Merge hotfix into develop" Merge made by the 'ort' strategy. patch.log 1 + 1 file changed, 1 insertion(+) create mode 100644 patch.log

2. Самостоятельное задание

Условие:

Нарисуйте схему Git Flow. Объясните, почему release сливается и в main, и в develop.

Выполнение (ход действий и результат):

```
$ git log --graph --oneline --all --decorate
* 9fc4732 (HEAD -> develop) Merge hotfix into develop
|\
* \ 0f804e4 Merge DB feature
|\
| * | fa639d4 (feature/db) Add DB config
|/
* | c255bcd Merge login feature
|\
```

```
| * | 73ad91b (feature/login) Add auth config
| / /
| | * f09c3db (tag: v1.0.1, main) Hotfix critical
| | /
| / /
| * 52da0e6 (hotfix/critical) Critical security fix
| /
* e5d96b0 Initial main
```

3. Ответы на контрольные вопросы

1. Вопрос 1:

Флаг --no-ff (no fast-forward) принудительно создает коммит слияния (merge commit). В Git Flow это необходимо для сохранения наглядной истории разработки: четко видно границы ветки (где она началась и где завершилась) и какие именно коммиты к ней относились. Это также позволяет легко отменить всю фичу целиком одной командой.

2. Вопрос 2:

Технически Git позволяет это делать, но это считается плохой практикой. Ветка develop является главной интеграционной веткой для всей команды. Прямые коммиты нарушают изоляцию кода, повышают риск сломать общую сборку проекта, делают невозможным проведение полноценного код-ревью через Pull Request и усложняют параллельную работу разработчиков.

3. Вопрос 3:

Если забыть слить hotfix в develop, возникнет регрессия ошибки (regression). Критический баг, который был экстренно исправлен на продакшене, останется в текущей ветке разработки. В результате, при выпуске следующего планового релиза этот баг снова попадет на продакшен, что полностью аннулирует проделанную работу по его исправлению.

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.